Frontend Interview Handbook — Part 8: Frontend System Design

Frontend system design questions are less about "the right answer" and more about your reasoning process. Always narrate: requirements → constraints → high-level approach → tradeoffs. Interviewers are grading your thinking, not pattern-matching to a memorized diagram.

SECTION A: HOW TO APPROACH ANY FRONTEND SYSTEM DESIGN QUESTION

Q1: What's a good general framework for answering "design X" questions (e.g., "design a news feed," "design an autocomplete search")?

Answer — use this structure every time:

1. **Clarify requirements** — functional (what must it do) and non-functional (scale, performance targets, device support, offline needs). Ask questions out loud even in a monologue format ("I'm assuming this needs to support mobile and handle ~10k items — let me know if that's off").
2. **Define the API/data contract** — what does the frontend need from the backend, what shape, pagination strategy.
3. **Component architecture** — break the UI into logical components, decide state ownership (what's local, what's shared).
4. **State management strategy** — server state (cache, fetching) vs client/UI state, and why.
5. **Performance considerations** — what's the bottleneck likely to be (list size, image loading, real-time updates) and how you'd address it.
6. **Edge cases** — loading, empty, error, offline, slow network, race conditions.
7. **Tradeoffs** — explicitly state what you're optimizing for and what you're giving up.

Why this matters most: In a live interview, silently designing in your head and presenting a finished diagram is a worse signal than talking through this structure out loud, even if you land on a similar final design — the process *is* the evaluation.

SECTION B: WORKED EXAMPLES

Q2: Design a typeahead/autocomplete search component.

Answer (talk through each point):

- **Requirements:** as user types, show matching suggestions with low latency; handle fast typing without flooding the network; handle no results, loading, and error states; keyboard navigation (arrow keys, enter, escape) for accessibility.
- **Data flow:** debounce input (e.g., 250-300ms) before firing a request — balances responsiveness against request volume. Cancel/ignore stale in-flight requests when a newer one fires (via `AbortController` or a library like React Query, which handles this by query key).
- **Component breakdown:** `SearchInput` (controlled input + keyboard handling), `SuggestionsList` (renders results, manages "active" highlighted index), a hook (`useAutocomplete`) encapsulating debounce + fetch + cancellation logic.
- **Performance:** cache recent queries client-side (avoid re-fetching identical strings the user backspaces into again); consider a small local prefix cache for near-instant results on common queries.
- **Edge cases:** empty query clears suggestions; error state shows a subtle inline message, not a blocking modal; empty results shows "no matches" rather than nothing (ambiguous vs "still loading").
- **Accessibility:** proper ARIA (`role="combobox"`, `aria-expanded`, `aria-activedescendant`) so screen reader users get equivalent functionality.

Q3: Design an infinite-scrolling feed (like a social media timeline).

Answer:

- **Requirements:** load more content as user scrolls near the bottom; handle very large total item counts without degrading performance; support pull-to-refresh or "new posts" indicator for content added since load.
- **Data fetching:** cursor-based pagination (not offset-based) — offset pagination breaks when items are inserted/removed between page fetches (classic bug: items shift, causing duplicates or skips); cursor-based (e.g., "give me items after ID X") stays consistent.
- **Rendering performance at scale: virtualization** (react-window/react-virtual) — only render DOM nodes for items currently in/near the viewport, not the entire list, since rendering thousands of DOM nodes directly would tank scroll performance and memory.
- **Trigger mechanism:** `IntersectionObserver` watching a sentinel element near the bottom of the list, rather than scroll event listeners (which fire far more often and require manual throttling).
- **State:** React Query's `useInfiniteQuery` (or equivalent) to manage pages, loading-more state, and caching cleanly.
- **Edge cases:** duplicate items if pagination cursor logic is off; layout shift if new items load above current scroll position (only append below, never prepend without explicit

"new posts" UI); handling the user scrolling faster than data loads (show a loading indicator, don't silently do nothing).

Q4: Design a real-time notification system (e.g., a bell icon with live updates).

Answer:

- **Requirements:** show new notifications without requiring a page refresh; indicate unread count; don't overwhelm the server with constant polling.
- **Transport options and tradeoffs:**
  - **Polling** (fetch every N seconds) — simplest to implement, but wasteful and has inherent latency; fine for low-urgency use cases.
  - **WebSockets** — persistent bidirectional connection, low latency, best for high-frequency real-time needs, but more infrastructure complexity (connection management, reconnection logic, scaling stateful connections).
  - **Server-Sent Events (SSE)** — simpler than WebSockets, one-directional (server→client only, which is often all you need for notifications), works over plain HTTP, auto-reconnects natively.
  - **Long polling** — middle ground, works where WebSockets/SSE aren't available.
  - **Recommendation for most notification use cases:** SSE — sufficient directionality, much simpler than WebSockets, and natively reconnects.
- **Frontend state:** keep unread count and notification list in a shared store/context (needed app-wide, likely in a persistent header); optimistically mark as read locally on click, sync to server in background.
- **Edge cases:** connection drops (reconnect strategy, and reconcile any notifications missed during the gap via a fetch-on-reconnect); very high notification volume (batch/debounce UI updates rather than re-rendering per single event).

Q5: Design a large data table/grid with sorting, filtering, and pagination (relevant to your DevExpress reporting experience).

Answer:

- **Requirements:** handle potentially large datasets, sort/filter without janky re-renders, support pagination or virtualization depending on total size.
- **Where should sorting/filtering happen — client or server?** Depends on data size: for small/bounded datasets (hundreds of rows), client-side is simpler and instant. For large datasets, push sorting/filtering to the backend (query params like `?sort=name&filter=status:active`) — client-side sorting of a huge dataset already fully loaded is itself a performance/memory problem, and usually the data shouldn't all be loaded client-side in the first place.

- **Rendering performance:** virtualize rows (render only visible rows) if the table can show many rows at once, even with server pagination, for smooth scroll within a page.
- **State:** URL-synced query params for sort/filter/page state (shareable/bookmarkable state, survives refresh) rather than only component state.
- **Edge cases:** loading state during sort/filter changes (avoid layout jump), empty-filtered-result state, handling filter+sort+pagination combined correctly on the backend query.

Q6: Design a form builder or a complex multi-step form (relevant to hotel booking/reservation forms from your SIP Consult experience).

Answer:

- **Requirements:** multiple steps, validation per step and on final submit, ability to go back without losing entered data, handle conditional fields (show/hide based on other answers).
- **State approach:** a form library (React Hook Form is common — uncontrolled-by-default for performance, less re-rendering than fully controlled forms) or a reducer holding the full form state across steps, with each step reading/writing its slice.
- **Validation:** schema-based validation (Zod/Yup) shared between client and ideally mirrored/enforced again server-side — never trust client-only validation for anything security or data-integrity relevant.
- **UX details worth mentioning:** persist in-progress form state (e.g., to `sessionStorage`) so a refresh/accidental navigation doesn't lose entered data; disable the submit button during submission to prevent double-submit; clear, specific inline error messages tied to the exact field, not a generic banner.

SECTION C: SYSTEM DESIGN CONCEPTS THAT COME UP REPEATEDLY

Q7: What is optimistic UI, and what's the tradeoff of using it broadly?

Answer: (Covered in Part 6 mechanically — here's the design-level framing.) Optimistic UI makes the app feel instant by updating the UI before server confirmation. Tradeoff: added complexity for rollback handling, and risk of showing the user something that turns out to be wrong (e.g., a "like" that actually failed due to a network error) — appropriate for low-stakes, easily-reversible actions; risky for anything financial or hard to undo cleanly.

Q8: How would you design a component library/design system to be used across multiple products?

Answer: Key considerations: consistent design tokens (colors, spacing, typography defined once, consumed everywhere — not hardcoded per component); components built with

composition in mind (accept `children`, avoid over-specific props that only fit one use case); accessibility baked in by default (not bolted on later); versioned and published as a package so consuming apps control their upgrade timing; Storybook or similar for isolated development/documentation/visual testing. Mention your Tailwind/shadcn/Ant Design experience here — you've worked with both a utility-first system and pre-built component libraries, so you understand the tradeoff between full design control (Tailwind) and speed/consistency (component libraries).

Q9: How would you architect a micro-frontend setup, and when is it actually worth the complexity?

Answer: Micro-frontends split a large app into independently deployable pieces (often by team/domain ownership), composed together at runtime or build time (via Module Federation, iframes, or server-side composition). Worth it when: multiple teams need to ship independently without coordinating releases, or different parts of a product have genuinely different tech needs. Not worth it for most teams/products — it adds real complexity (shared dependency versioning, consistent UX across independently-built pieces, cross-app communication) that a well-organized monorepo/modular single app avoids. Good instinct to show: default to NOT recommending micro-frontends unless the org-structure/scale problem genuinely calls for it — over-engineering is a real interview red flag.

Rapid-Fire Round

- **Q: Client-side vs server-side search — how do you decide?** A: Small, bounded dataset → client-side instant filtering. Large or full-text/fuzzy search → server-side (or a dedicated search service like Elasticsearch/Algolia).
- **Q: How would you handle a component that needs data from 3 different unrelated APIs?** A: Fetch in parallel (`Promise.all` or parallel React Query hooks), not sequentially, unless one genuinely depends on another's result — sequential awaiting when unnecessary is a common performance mistake.
- **Q: What's the tradeoff of putting business logic in custom hooks vs a state management store?** A: Hooks keep logic colocated and easy to reason about per-feature; a store centralizes cross-cutting logic but can become a dumping ground if not organized by domain/slice.
- **Q: How would you design for offline support?** A: Service worker caching strategy (cache-first for static assets, network-first with cache fallback for data), local queue for actions taken offline to sync once reconnected, clear UI indicating offline state.

End of Part 8. Say "next part" for Part 9: Behavioral, Company Fit & Mock Interview.